

10

Lecture 10

Process Synchronization I: Critical Section and Race Conditions

by - Brijesh Shukla
C07: Operating Systems

Join the lecture online on your dashboard

Today's Agenda

- **Introduction to Process Synchronization**
- **The Critical Section Problem**
- **Race Conditions: Examples and Consequences**
- **Requirements for Synchronization Solutions**
- **Concurrency in Modern Systems**
- **Preview of Synchronization Techniques**



Introduction to Process Synchronization

What is Process Synchronization?

A mechanism that ensures multiple concurrent processes or threads can safely share resources and coordinate their execution without conflicts.

Why Do We Need It?

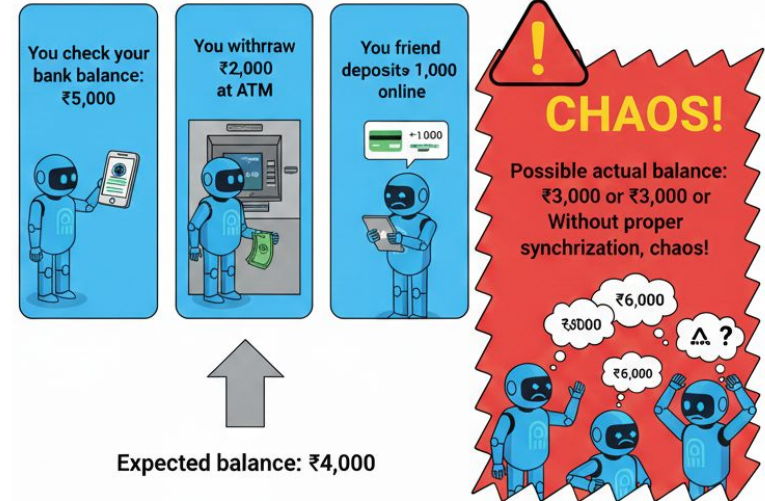
Real-World Scenario:

- You check your bank balance: ₹5,000
- You withdraw ₹2,000 at ATM
- Your friend deposits ₹1,000 online

Expected balance: ₹4,000

Possible actual balance: ₹3,000 or ₹6,000. Without proper synchronization, chaos!

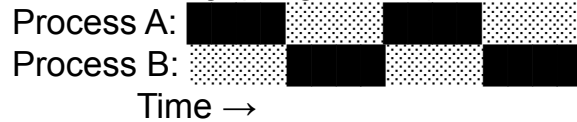
Real-World Scenario



Concurrent vs Parallel Execution

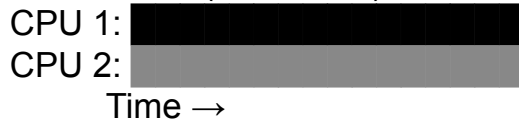
Two Ways to Run Multiple Processes

Concurrency (Single CPU)



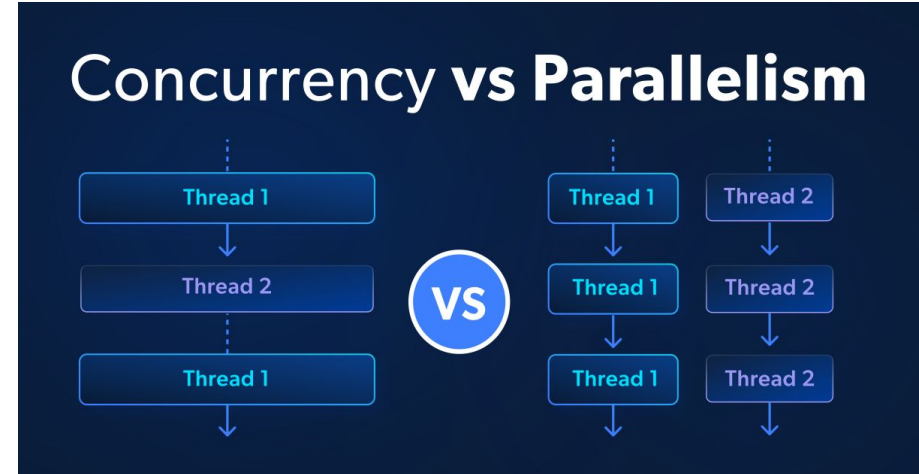
Interleaving - taking turns rapidly

Parallelism (Multi-CPU)



Truly simultaneous execution

Both need synchronization!



The Critical Section Problem

What is a Critical Section?

A critical section is a segment of code where a process accesses shared resources (variables, files, data structures) that must not be accessed by more than one process at a time.

The Danger Zone ⚠️

```
do {
  // 🚪 Entry Section - request permission

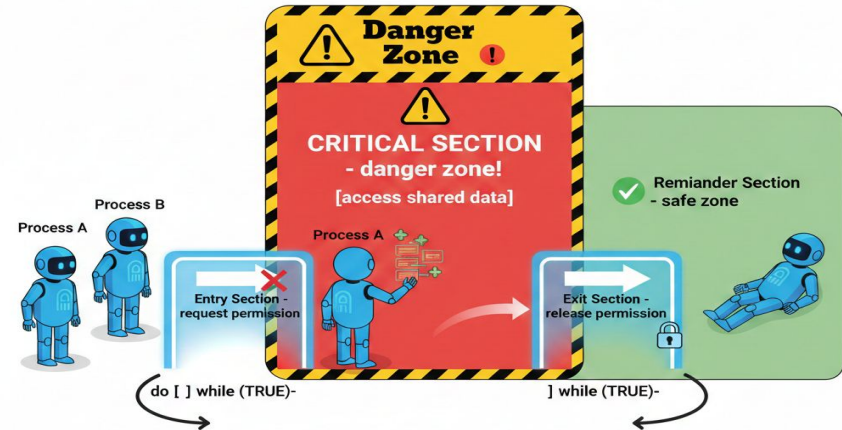
  // ⚠️ CRITICAL SECTION - danger zone!
  [access shared data]

  // 🚪 Exit Section - release permission

  // ✅ Remainder Section - safe zone

} while (TRUE);
```

Critical Section Explained



**Goal: Only ONE process
in critical section a time**

Critical Section Example

CRITICAL SECTION - Multiple threads access this!

```
int balance = read_balance(); // 📖 Read  
balance = balance - amount;   // 🧮 Calculate  
write_balance(balance);      // ✍️ Write
```

Why is this critical?

- Multiple users → Same account
- Simultaneous transactions
- Shared variable: balance



Bank Database



Read Balance



Calculate New
Balance



Multiple Users



Simultaneous
Transactions



Shared
Account

Race Condition – Concept

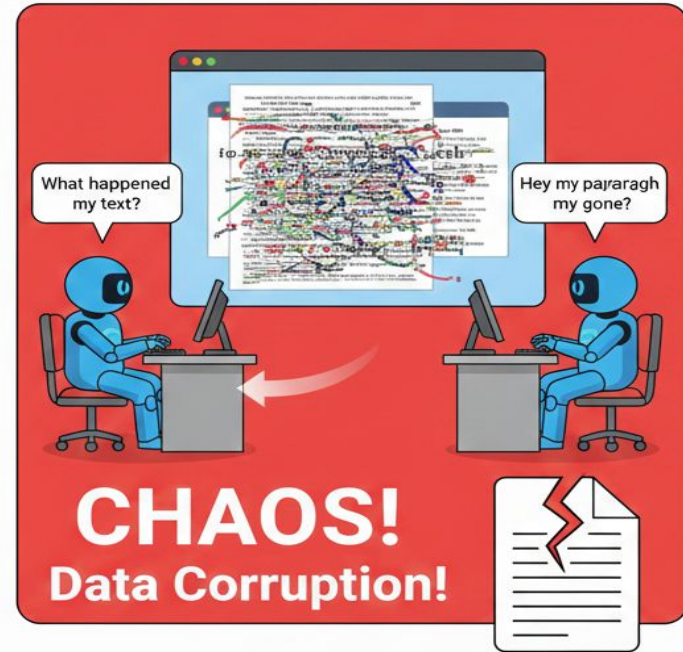
What Is It?

Outcome depends on execution timing

Key Characteristics:

-  Non-deterministic (unpredictable)
-  Timing-dependent
-  Hard to reproduce & debug
-  Can corrupt data

Analogy: *Two people editing a Google Doc without seeing each other's changes*



Race Condition – Classic Example (Bank Account)

The Bank Account Problem:

Consider a shared bank account with balance = 5000 Two processes execute simultaneously:

- Process A: Deposits 1000
- Process B: Withdraws 2000

For **Deposit** (Process A):

```
int balance = read_balance(); // reads 5000
balance = balance + 1000;      // calculates 6000

write_balance(balance);        // writes 6000
```

For **Withdrawal** (Process B):

```
int balance = read_balance(); // reads 5000
balance = balance - 2000;      // calculates 3000
write_balance(balance);        // writes 3000
```

Expected Final Balance: 4000 (5000 + 1000 - 2000)

Race Condition – Execution Scenarios

Scenario 1: Sequential Execution (Correct)

Time	Process A	Process B	Balance
t1	read(5000)		5000
t2	calc(6000)		5000
t3	write(6000)		6000
t4		read(6000)	6000
t5		calc(4000)	6000
t6		write(4000)	4000

✓ **Result: 4000 (Correct)**

Scenario 2: Interleaved Execution (Race Condition)

Time	Process A	Process B	Balance
t1	read(5000)		5000
t2		read(5000)	5000
t3	calc(6000)		5000
t4		calc(3000)	5000
t5	write(6000)		6000
t6		write(3000)	3000

✗ **Result: 3000 (Incorrect! Lost 1000)**

Race Condition – Producer-Consumer Example

Bounded Buffer Problem Setup:

Shared buffer with counter variable tracking number of items.

Producer Code:

```
while (true) {  
    // produce item  
    while (counter == BUFFER_SIZE)  
        ; // buffer full, wait  
    buffer[in] = item;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++; // CRITICAL OPERATION  
}
```

Consumer Code:

```
while (true) {  
    while (counter == 0)  
        ; // buffer empty, wait  
    item = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--; // CRITICAL OPERATION  
}
```

***The Problem:** counter++ and counter-- are NOT atomic operations!*

Why counter++ is Dangerous

Looks Simple, Right?

counter++; // Just one line!

But In Machine Code Assembly

```
LOAD    R1, counter    // Step 1: Read
ADD     R1, R1, 1       // Step 2: Calculate
STORE   R1, counter     // Step 3: Write
```

***3 separate instructions = 3 points where
interruption can occur!***

The Scenario:

Initial: counter = 5

Thread 1: counter++ (wants: 6)

Thread 2: counter++ (wants: 7)

✗ Race Condition Execution

T1: LOAD R1, counter	(R1 = 5)
T2: LOAD R2, counter	(R2 = 5)
T1: ADD R1, R1, 1	(R1 = 6)
T2: ADD R2, R2, 1	(R2 = 6)
T1: STORE R1, counter	(counter = 6)
T2: STORE R2, counter	(counter = 6)

Expected: 7

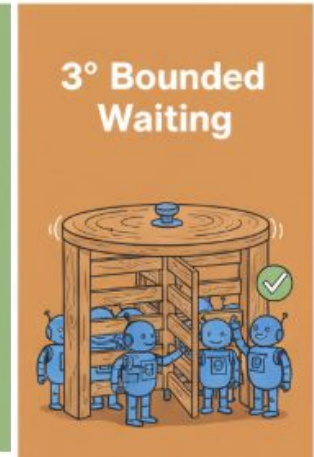
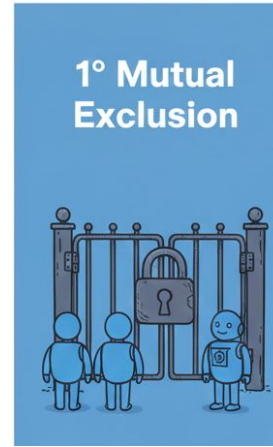
Actual: 6

One increment lost!

Requirements – Critical Section Solutions

Every Solution **MUST** Satisfy All Three:

- 1 Mutual Exclusion
> Only ONE process in critical section at a time
- 2 Progress
> If no one's inside, someone waiting **MUST** get in
- 3 Bounded Waiting
> Everyone gets their turn eventually (no starvation)



Memory Aid: MPB (Mutual, Progress, Bounded)

Requirement 1 – Mutual Exclusion

The Bathroom Rule

Only ONE person inside at a time

Process P1: [inside critical section]

Process P2: [must wait] 

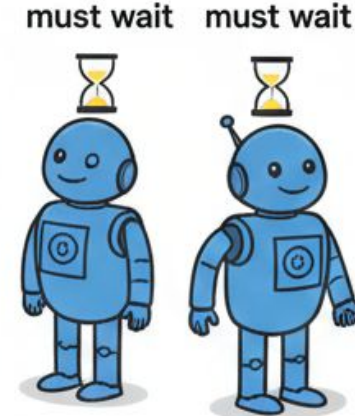
Process P3: [must wait] 

Why It Matters

- Prevents simultaneous access
- Protects data integrity
- Most fundamental requirement

🎯 What happens if this is violated?

Answer: Data corruption!



Requirement 2 – Progress

No Artificial Delays

Bad Scenario:

Critical Section: [EMPTY]

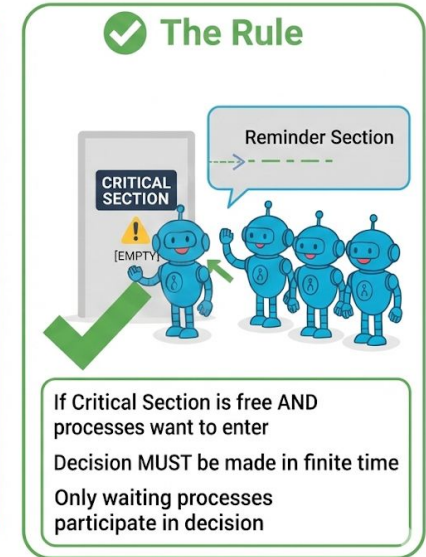
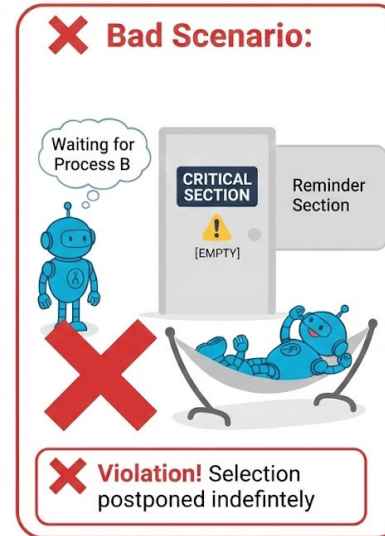
Process A: Waiting for Process B

Process B: In remainder section (not even interested!)

✗ Violation! Selection postponed indefinitely

The Rule

- If Critical Section is free AND processes want to enter
- Decision MUST be made in finite time
- Only waiting processes participate in decision



Requirement 3 – Bounded Waiting

Fairness Guarantee

Bad Scenario:

Process A: Requested entry 1,000 times

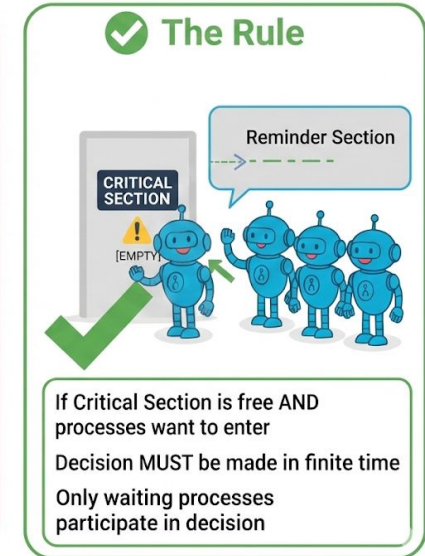
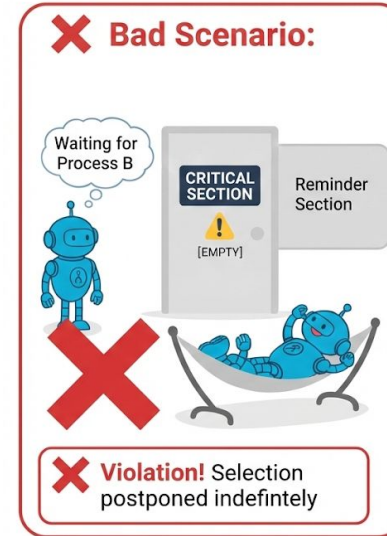
Process B: Keeps entering first

Process A: Still waiting... (STARVATION)

The Rule

There exists a **limit** on how many times others can enter before you

Not Necessarily FIFO, but no infinite waiting!



Additional Desirable Properties

Beyond the Three Main Requirements

⚡ Performance

- Low overhead
- Fast entry
- Scales well

⚖️ Fairness

- No starvation
- FIFO access
- No priority inversion

🧩 Simplicity

- Easy to understand
- Fewer bugs
- Maintainable

🌐 Portability

- Hardware-independent
- No timing assumptions

Performance:



Fairness



Simplicity



Portability



Real-World Consequences of Race Conditions

Historical Examples

⚠️ Therac-25 (1985–87)

- Control software race condition
- Lethal radiation overdoses
- **Cost:** Human lives

⚠️ Northeast Blackout (2003)

- Race condition in alarm system
- 50 million people affected
- **Cost:** ~\$6 billion

Modern Examples

- 🏦 Banking → incorrect balances
- 🛒 E-commerce → oversold inventory
- 🎮 Gaming → item duplication glitches
- 📱 Social media → data corruption

Why These Happen

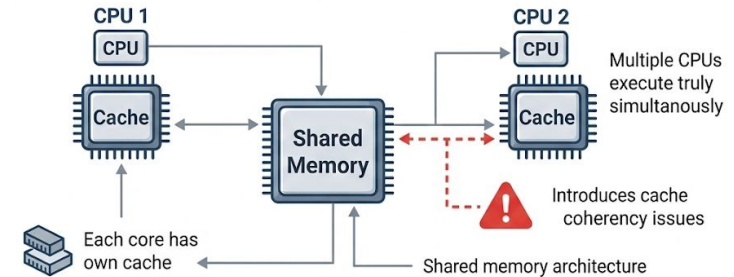
- 🔄 Concurrent access to shared resources
- 🕒 Timing-dependent behavior
- 🔧 Hard to test & reproduce

Historical Examples:	Modern Examples:
 Therac-25 Radiation Therapy Machine (1985-1987)  Cost: Human lives	 Banking Systems
Northeast Blackout (2003)  Cost: \$6 Billion	 E-commerce
	 Social Media

Concurrency in Modern Systems

Multi-Core Processors:

- Multiple CPU cores execute truly simultaneously
- Shared memory architecture
- Each core has own cache (introduces cache coherency issues)
- Requires sophisticated synchronization



Challenges in Modern Systems:

-  **Cache coherency:** Each core has own cache
-  **Memory ordering:** Instructions can be reordered
-  **Scalability:** More cores = more contention

Synchronization is MORE critical now!

Challenges in Modern Systems



Cache Coherency



Memory Ordering



Scalability

Where OS Uses Synchronization

Everywhere!

Process Management

- Scheduling queues
- Process tables

Memory Management

- Page tables
- Free frame lists

File Systems

- Inode tables
- Directory structures

Device Management

- Device status tables
- I/O request queues



Every shared data structure needs protection!

Preview of Synchronization Techniques

We Know the Problem... Now for Solutions!

Software Solutions

-  Peterson's Algorithm (2 processes)
-  Dekker's Algorithm (n processes)

Hardware Support

-  Test-and-Set
-  Compare-and-Swap
-  Mutex Locks

High-Level Constructs

-  Semaphores
-  Monitors

Modern Approaches

- Lock-free structures
- Transactional memory

Practice Time!

Problem 1: Identify the Critical Section

```
int shared_counter = 0;

void increment() {
    for (int i = 0; i < 1000; i++) {
        shared_counter++;
    }
}
```

Shared Resource: `shared_counter`
(global variable)

Critical Section: The line
`shared_counter++;`

Problem 2: Thread Operations on Counter

Given:

- Two threads execute on shared variable
- Thread 1: `counter++`
- Thread 2: `counter--`
- Initial value: `counter = 10`

Question: What are ALL possible final values?

Final Value	Correct?	How it happens
9	✗	Thread 2 writes last, overwrites T1's increment
10	✓	Sequential execution (either order)
11	✗	Thread 1 writes last, overwrites T2's decrement

Key Takeaways

What You Must Remember

- Critical sections = code accessing shared resources
- Race conditions = timing-dependent bugs (DANGEROUS!)
- Three requirements = MPB (all three!)
 - Mutual Exclusion
 - Progress
 - Bounded Waiting
- Simple solutions fail - we need sophisticated mechanisms
- Modern systems make this harder but more important



Thank You!

Thank you for your attention. Feel free to ask questions or revisit the topics covered.